

LangChain - Tools

⇒ Only LLM

⇒ LLM + external tools.

In LangChain, tools are callable functions with well defined inputs & outputs.

Query
↓

LLM decides: Can I answer directly?

↓

Which Tool to call?

↓

Application calls the tool.

↓

Tool result is sent back to the LLM.

↓

LLM gives user-friendly response.

→ tool decorator.

@tool

```
def fetch_from_db():
```

```
    _____  
    _____  
    _____  
    _____  
    _____
```

@tool

```
def get_order_status(order_id: str):
```

// Make a DB call & get
the order status for the
given order_id.

Tool quality is very important in production
system.

→ Tools should have proper input & output
schemas defined.

→ Tool names are generally written in
snake-case format.

get_order_status.

Create_refund_ticket

Input | Output Schema.

Pydantic

```
class RefundInput ( BaseModel ):
    """
    """
```

Order-Id: $\text{Str} = \text{Field}(_, _, _, _)$

```
@tool (args_schema=RefundInput)
```

```
def create_refund_ticket (order_id, ---):
```

⇒ bind-tools

```
model.bind_tools([_, _, _, _, _])
```

Error - Containment Patterns.

1) Return structured error objects.

Bad tool output:

```
return "Something went wrong"
```

Better:

```
return json.dumps({
    "ok": False,
    "error_type": "ORDER_NOT_FOUND",
    "message": "No order found for ORD-9999"
})
```

2) Catch tool exceptions.

Bad:

```
tool_result = selected_tool.invoke(tool_call)
```

If the tool fails, the whole app crashes.

Better:

```
try:
    tool_result = selected_tool.invoke(tool_call)
except Exception as error:
    tool_result = ToolMessage(
        content=json.dumps({
            "ok": False,
            "error_type": error.__class__.__name__,
            "message": str(error)
        }),
        tool_call_id=tool_call["id"]
    )
```

3) Validate args | input params before tool calling

4) Keep the tools minimal & deterministic.

5) Limit tool output size.

Diagnosing tool-selection & argument faults.